

\*具体代码请参考“代码”文件夹中的“real1.py”（处理原始语料库）和“real2.py”（处理改进的语料库）

代码处理具体思路：

### （1）hmm 训练和评估的思路

对于原始的语料库，首先读入数据，并且构造 hmm 模型。将语料库中的数据按照 9: 1 的比例分为训练集和测试集（根据实际情况可以改变比例），用训练集对 hmm 模型进行训练（为测试，也使用了全部数据进行训练），之后对标注的准确率和召回率。

特殊的，我们也可以计算句子的准确率和召回率，我们定义一个句子标注正确指的是句子中多有分词的标注都正确，由于对于句子，只有正确和不正确两种区分，所以句子的召回率意义不大，结果都为 100%（但还是在代码中进行计算了）。

### （2）改变语料库思路

语料库中有一万多个句子，一个个去修改标注显然不现实，利用其它的标注器来标注也比较难操作，这里用比较简单的方法。

因为介词后是名词和名词短语，介词后是一个完整的句子（虽然也存在一个词既是连词也是介词的情况），所以以此为依据，如果原标注 IN 后面的标注是 DT（冠词）、NNP（专有名词）、NN（普通名词）、NNS（名词复数）、CD（数字）、VBG（动词进行时）或 JJ（形容词），则标注改为 IN-prep，否则改为 IN-conj。

当然，这个方法的标注并不准确。

测试结果比较：

对于原始的语料库，准确率、召回率、句子的准确率、句子的召回率分别如下：

```
0.5734858204663098
0.9555555555555556
0.2084006462035541
1.0
```

对于修改后的语料库，准确率、召回率、句子的准确率、句子的召回率分别如下：

```
0.5629783657036677
0.9565217391304348
0.20463112547119008
1.0
```

可见，准确率和句子的准确率都有所降低，召回率有所升高

考虑到可能是训练样本的不足，接下来使用全部的数据进行训练，对于原始的语料库，有：

```
0.9732115205946114
0.9777777777777777
0.5455035002692514
1.0
```

对于修改后的语料库，有：

```
0.9721939565544396
0.9782608695652174
0.5444264943457189
1.0
```

可见，仍然是准确率和句子的准确率都有所降低召回率有所升高。

结果分析与结论：

可见准确率有所降低，召回率有所升高，但是差别并不明显。

可能是一下一些原因：

（1）修改后的标注不准确；（2）训练样本太少；（3）标签增多造成的影响（可以看出在使用我的修改标注方法时，IN-prep 是占多数的）

初步试验的结论是，老师的假设并没有大幅提高准确率，甚至有所降低，但是如果是大量训练和进行改进后，也许能够得到更好的效果。